

Vision Day4 hw1 AI 보고서

로봇 21기 예비단원 최윤서

1. 카메라 frame 받아오기 실패 이슈

Day1 hw1을 진행할 때 발생했던 이슈가 똑같이 발생했습니다.
하드웨어 문제라는 것을 알았지만 ubuntu환경을 재설치했기 때문에 Day1때 사용했던 코드를 적용할 수 없어서 AI에게 다시 물어봤습니다. 전체 내용을 모두 첨부하기엔 양이 너무 많아서 카메라 문제를 수정한 과정을 요약해달라고했고 다음 내용과 같습니다.

문제 요약

증상: `cv::VideoCapture` 로 웹캠을 열려고 했는데, `cap_.open(0)` 은 성공하지만 (`isOpened() == true`) 실제 프레임 읽으면 (`cap_ >> frame`) 계속 빈 프레임만 나온다.

근본 원인: 이 노트북(Samsung Galaxy Book4 Pro, 940XGK)의 내장 웹캠은 일반 USB 웹캠(UVC)이 아니라 **Intel IPU6 + MIPI CSI-2 카메라**였습니다. 이 방식은 진단해 나가면서 두 겹의 문제가 있었어요.

1. **커널 드라이버 구조 문제:** IPU6는 raw 센서 데이터를 그대로 내보내는 방식이라, `v4l2-ctl --list-devices` 로 확인했을 때 하나의 카메라가 `/dev/video0` 부터 `/dev/video47` 까지 **48개의 노드**로 쪼개져 있었고, 이 중 색상 이미지를 바로 주는 노드는 없었습니다. (일반 `cv::VideoCapture` 가 처리 못 하는 raw Bayer 데이터)
2. **센서 드라이버 버그:** `dmesg` 로 확인해보니 실제로는 센서 자체는 정상 인식됐지만, `ov02c10` 커널 드라이버가 이 보드의 센서 클럭(26MHz)을 지원 목록에 안 넣어놔서 probe 자체가 실패하고 있었습니다 (`external clock 26000000 is not supported`). 이걸 하드웨어 결함이 아니라 순수하게 **리눅스 드라이버 소스의 누락**이었고, Windows에서는 인텔/삼성에서 제공하는 정식 드라이버가 이 문제를 이미 처리해놔서 문제없이 잘 됐던 거예요.

어떻게 고쳤나

코드를 고쳐서 해결한 게 아니라, **리눅스 드라이버/유저스페이스 스택 자체를 교체**해서 해결했습니다.

`Andycodeman/samsung-galaxy-book-linux-fixes` 저장소의 `webcam-fix-libcamera` 설치 스크립트를 실행해서:

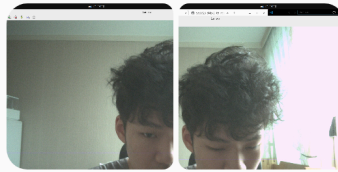
1. 26MHz 클럭을 지원하는 **패치된 ov02c10 DKMS 드라이버**로 교체
2. IPU6의 raw 노드들을 처리해서 실제 컬러 이미지로 변환해주는 **libcamera** 스택을 소스에서 빌드 및 설치
3. 지저분한 48개의 raw IPU6 노드는 숨기고, 대신 ****camera-relay****가 libcamera로부터 받은 완성된 영상을 `/dev/video0` 이라는 하나의 깔끔한 v4l2loopback 가상 장치로 내보내도록 구성

결과적으로 애플리케이션(우리 ROS2 코드 포함) 입장에서는 `/dev/video0` 을 여느 평범한 UVC 웹캠처럼 열면 되는 상태가 됐고, 그래서 **코드 자체는 cv::CAP_V4L2 백엔드 명시 정도만 추가**하고 원래 구조 그대로 정상 작동하게 된 겁니다.

정리하면: **코드 문제가 아니라 이 노트북 모델 특유의 리눅스 드라이버 미비 문제**였고, 해결책은 알려진 커뮤니티 패치로 드라이버/카메라 스택 자체를 고치는 것이었습니다.

2. 카메라 화면 크기 조절 및 GUI 기능 최소화

카메라 화면을 띄웠을 때 너무 확대돼서 출력됐습니다.
출력되는 화면 사이즈를 어떻게 줄여야할지 AI에게 물어봤습니다.



이렇게 뜨는데 화면 크기 조절이 안됨

맞습니다. 지금 스크린샷을 보면 카메라 영상 자체의 해상도는 1920×1080인데, OpenCV 창이 영상 크기에 맞춰지는 `WINDOW_AUTOSIZE` 상태입니다.

현재 코드의:

```
</> C++  
cv::imshow("Camera", frame);
```

만 사용하면 OpenCV가 기본적으로 영상 크기에 맞춰 창을 관리하기 때문에 창 크기를 자유롭게 조절하기 어렵습니다.

1단계: OpenCV 창을 자유롭게 조절 가능하게 변경

`cv::imshow()` 바로 전에 다음 한 줄을 추가하세요.

```
</> C++  
cv::namedWindow("Camera", cv::WINDOW_NORMAL);  
cv::imshow("Camera", frame);
```

즉 기존:

```
</> C++  
cv::imshow("Camera", frame);
```

을

```
</> C++  
cv::namedWindow("Camera", cv::WINDOW_NORMAL);  
cv::imshow("Camera", frame);
```

으로 바꾸면 됩니다.

다만 `namedWindow()` 를 `while` 안에서 매 프레임 호출하는 것은 좋지 않습니다. 따라서 더 정확하게는 `while` 전에 한 번만 만들어야 합니다.

현재 코드에서:

```
<> C++  
  
cv::Mat frame;  
  
while (rcLcnp::ok())
```

를 다음처럼 바꾸세요.

```
<> C++  
  
cv::Mat frame;  
  
cv::namedWindow("Camera", cv::WINDOW_NORMAL);  
  
while (rcLcnp::ok())
```

그리고 기존의

```
<> C++  
  
cv::imshow("Camera", frame);
```

는 그대로 둡니다.

이것만 수정하고 실행해보세요.

창 크기를 마우스로 조절할 수 있는지 확인하면 됩니다.

그리고 중요한 점은, 이 수정은 창 크기만 조절하는 것이고 카메라의 실제 화각(줌)을 바꾸는 것은 아닙니다. 창 크기가 조절된 다음에도 영상 자체가 확대되어 있다면 그때는 Camera Relay/libcamera 쪽의 FOV/crop 문제를 따로 확인하겠습니다.

잘 뜨는데 rgb 값이랑 내 마우스 커서 위치는 왜 뜨는거야?

그건 카메라 영상에 RGB 값이나 마우스 좌표를 우리가 추가한 게 아닙니다.

지금 보이는 RGB 값과 마우스 위치 정보는 OpenCV의 HighGUI 창이 Qt 백엔드로 실행되면서 제공하는 기능입니다.

왜 생겼냐면

우리가 방금 추가한:

```
<> C++  
  
cv::namedWindow("Camera", cv::WINDOW_NORMAL);
```

때문에 OpenCV가 일반적인 이미지 출력만 하는 게 아니라 Qt 기반의 조절 가능한 GUI 창을 사용하고 있습니다.

그래서 창 상단에 보이는:

- 확대/축소 버튼
- 창 크기 조절 기능
- 마우스가 가리키는 픽셀 위치
- 해당 픽셀의 RGB 값

같은 OpenCV GUI 기능이 같이 나타나는 것입니다.

특히 마우스를 영상 위에 올리면 현재 가리키는 픽셀의 정보를 표시합니다.

중요한 점

이것은 카메라 영상 자체에 포함된 정보가 아닙니다.

즉 YOLO에 프레임을 넘길 때는:



```
<> C++
frame
```

그대로 전달되고, RGB/좌표 표시 같은 GUI 정보는 포함되지 않습니다.

현재 상태에서는 그대로 두어도 과제 수행에는 문제가 없습니다.

오히려 지금 우리가 확인한 것은:

```
카메라 실제 해상도 → 1920 × 1080
OpenCV 수신 → 1920 × 1080
창 크기 → 자유롭게 조절 가능
```

까지 정상적으로 된 것입니다.

원하면 다음 단계에서 **> 상단의 OpenCV GUI 버튼과 RGB/좌표 표시 없이 깔끔한 카메라 영상만 띄우는 방법**을 적용할 수 있습니다.

📄 📄 📄 📄 ...

상단의 OpenCV GUI 버튼과 RGB/좌표 표시 없이 깔끔한 카메라 영상만 띄우는 방법 알려줘

가능합니다. 다만 지금 보이는 상단 버튼과 RGB/좌표 표시를 완전히 없애려면 OpenCV HighGUI 창을 쓰지 않고, 프레임 자체를 별도의 GUI로 표시하는 방식이 필요합니다.

그런데 현재 과제는 ROS2 + C++ + OpenCV이고, 나중에 UI 노드로 연결할 예정이므로 지금 단계에서는 굳이 Qt를 다시 붙일 필요가 없습니다.

우선 OpenCV 창에서 상단 도구 영역 없이 영상만 깔끔하게 표시하려면 `cv::WINDOW_GUI_NORMAL` 을 사용해볼 수 있습니다.

현재:

```
<> C++
cv::namedWindow("Camera", cv::WINDOW_NORMAL);
```

을

```
<> C++
cv::namedWindow("Camera", cv::WINDOW_NORMAL | cv::WINDOW_GUI_NORMAL);
```

로 변경하세요.

WINDOW_GUI_NORMAL 은 OpenCV의 GUI 확장 기능을 최소화하는 옵션입니다.

다만 여기서 한 가지 구분해야 합니다.

- 창 크기 조절 가능 → `WINDOW_NORMAL`
- OpenCV의 GUI 기능 최소화 → `WINDOW_GUI_NORMAL`
- 운영체제 창의 제목 표시줄까지 완전히 제거 → OpenCV `imshow()` 만으로는 제한적

따라서 먼저 위 한 줄만 바꾸고 실행해보세요.

```
<> Bash
cd ~/ros2_ws
colcon build --packages-select pytorch_object_detection
source install/setup.bash
ros2 run pytorch_object_detection camera_node
```

상단 버튼과 RGB/좌표 표시가 없어지는지 확인해보면 됩니다.

느낀점)

1번은 Day1 때 사용했던 방법인데 다시 기억해서 사용하지 못한 것이 살짝 아쉬웠습니다. 이번 과제를 통해 하드웨어와 운영체제 환경에 따라 동일한 코드도 다르게 동작할 수 있다는 것을 다시 한번 느낄 수 있었습니다.